

Guía de Ejercicios 2

Tecnología Digital VI

Juan Ignacio Elosegui

Universidad Torcuato Di Tella
Abril 2026

Modelos Lineales

Ejercicio 1

- Y_i es la variable dependiente.
- β_0 es el intercepto.
- β_1 es la pendiente del atributo.
- X_i es la observación i del *feature* X .
- ϵ_i es el error aleatorio de la observación i .

Ejercicio 2

No es lo óptimo usar *gradient descent*, porque es un proceso iterativo. Si querés, podés usarlo igual. Lo mejor para resolver un problema de regresión lineal es usar mínimos cuadrados, porque tiene una fórmula cerrada. Seguramente haya excepciones, pero en general conviene la segunda opción.

Ejercicio 3

Sigue siendo un modelo lineal porque se usa la combinación lineal de los atributos almacenada en la variable z en la función sigmoide. La sigmoide solamente es una transformación que se aplica sobre esa combinación lineal.

Ejercicio 4

A ver qué pasa si la combinación lineal es cero, es decir:

$$\begin{aligned} z &= 0 \\ \Rightarrow \sigma(0) &= \frac{1}{1 + e^{-0}} \\ \Rightarrow \sigma(0) &= \frac{1}{1 + 1} \\ \Rightarrow \sigma(0) &= \frac{1}{2} \\ \therefore \sigma(0) &= 0,5 \end{aligned}$$

La probabilidad asignada da 0,5. Esto quiere decir que la clasificación binaria resultante depende de dónde esté ubicada la frontera de decisión.

Ejercicio 5

Para optimizar una regresión logística se suele usar *Log-Loss* porque es continua y derivable, lo que lo hace genial para usar el método de *gradient descent*.

La métrica de evaluación es *Accuracy* -claramente está también *Precision*, *Recall*, etcétera- las cuales no son derivables en todo su dominio, por lo que no sirve para *gradient descent*.

Aunque son distintas, ambas persiguen el mismo objetivo: cuando la *Log-Loss* baja, la *Accuracy* tiende a subir. Minimizar la incertidumbre probabilística del modelo termina mejorando las clasificaciones.

Supongamos dos modelos: A y B.

- **Modelo A.** Predijo las probabilidades $\{0,9, 0,8, 0,9, 0,95\}$, que son todas clasificadas como clase 1. Este modelo tiene un *Accuracy* de 100 %.
- **Modelo B.** Predijo las probabilidades $\{0,5, 0,55, 0,6, 0,5\}$, que son todas clasificadas como clase 1. Este modelo tiene un *Accuracy* de 100 % también.

Tienen la misma *Accuracy* los dos modelos, pero lo que hace la *Log-Loss* es -básicamente- calcular qué tan seguro está el modelo de la clasificación que le puso a una observación. Es como una métrica de calidad de la clasificación. El *Log-Loss* del Modelo A es mucho menor porque sus probabilidades están más cerca de su clase real (1) que el Modelo B.

Ejercicio 6

Una Regresión Logística estándar no puede separar este dataset de manera efectiva, ya que su frontera de decisión es un hiperplano, y no existe ninguna recta capaz de separar un círculo interior de un anillo exterior. Pero, mediante ingeniería de atributos, es posible lograrlo.

Si se introduce un atributo $X_3 = X_1^2 + X_2^2$, este representa la distancia al origen al cuadrado. De esta forma, en el espacio expandido de atributos (X_1, X_2, X_3) , las clases se vuelven linealmente separables y la regresión logística puede encontrar un umbral sobre X_3 que los separe correctamente.

El modelo sigue siendo lineal en sus parámetros $(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3)$, pero al operar sobre atributos con transformaciones no lineales, logra capturar una frontera que en el espacio original es circular.

Árboles de Decisión

Ejercicio 7

En un **modelo lineal**, la frontera de decisión es un único hiperplano que divide todo el espacio de atributos en dos regiones. Esta frontera puede tener cualquier orientación, es decir, puede ser diagonal a los ejes.

En un **árbol de decisión**, este divide el espacio de atributos en rectángulos tras hacer cortes de manera sucesiva de a un atributo por vez. Las fronteras son paralelas a los ejes.

Ejercicio 8

- **Para datos faltantes.** El árbol puede usar **surrogate splits**, que funcionan como *divisiones sustitutas*. Cuando el árbol encuentra que para una observación falta el valor de la variable por la que quiere partir, busca otra variable que produzca una partición *similar* y usa esa en su lugar.
- **Para variables categóricas.** En lugar de necesitar *One-Hot Encoding*, el árbol puede partir directamente evaluando *subconjuntos* de categorías. Por ejemplo, si tenemos una variable **Color** con valores {Rojo, Azul, Verde}, el árbol puede *evaluar particiones* como {Rojo, Azul} vs {Verde}, probando distintas agrupaciones de categorías y *eligiendo la que mejor separe las clases*.

Ejercicio 9

Un índice de *Gini* igual a cero implica que la pureza de ese nodo es *total*, es decir, todas las observaciones dentro del nodo son de la misma clase.

El índice Gini se calcula como:

$$Gini = 1 - \sum_{i=1}^C p_i^2$$

donde p_i es la proporción de muestras de la clase i en el nodo y C es la cantidad de clases. Si todas las muestras pertenecen a una misma clase, entonces una proporción vale 1 y las demás valen 0, por lo que

$$Gini = 1 - 1^2 = 0$$

En el extremo opuesto, si se tienen dos clases perfectamente mezcladas en proporciones 50/50,

$$Gini = 1 - (0,5^2 + 0,5^2) = 0,5$$

que es el valor máximo de impureza para clasificación binaria.

Intuitivamente, el índice Gini mide la **probabilidad de clasificar incorrectamente una muestra si se la etiquetara al azar según la distribución de clases del nodo**. Si todas las muestras son de la misma clase, esa probabilidad es cero.

Ejercicio 10

Consideremos un nodo padre con 200 muestras: 100 de clase A y 100 de clase B (proporción 50/50).

Se evalúa el siguiente split:

- Hijo izquierdo: 60 de clase A, 40 de clase B (100 muestras)
- Hijo derecho: 40 de clase A, 60 de clase B (100 muestras)

El **Missclassification Error** se calcula como $1 - \max(p_i)$, es decir, la proporción de la clase que no es mayoritaria.

$$ME_{\text{padre}} = 1 - \max\left(\frac{100}{200}, \frac{100}{200}\right) = 1 - 0,5 = 0,5$$

$$ME_{\text{izq}} = 1 - \max\left(\frac{60}{100}, \frac{40}{100}\right) = 1 - 0,6 = 0,4$$

$$ME_{\text{der}} = 1 - \max\left(\frac{40}{100}, \frac{60}{100}\right) = 1 - 0,6 = 0,4$$

$$ME_{\text{hijos}} = \frac{100}{200} \times 0,4 + \frac{100}{200} \times 0,4 = 0,4$$

Misclassification Error baja de 0,5 a 0,4. Ahora bien, supongamos que el *split* anterior no está disponible y el mejor *split* posible genera:

- Hijo izquierdo: 50 de clase A, 50 de clase B (100 muestras)
- Hijo derecho: 50 de clase A, 50 de clase B (100 muestras)

$$ME_{\text{izq}} = 1 - 0,5 = 0,5; \quad ME_{\text{der}} = 1 - 0,5 = 0,5$$

$$ME_{\text{hijos}} = 0,5$$

En este caso, *Misclassification Error* no detecta ningún progreso (se mantiene en 0,5), y el árbol no puede avanzar.

Ahora, con el **índice Gini**, volvemos al primer *split* (60A/40B vs 40A/60B):

$$Gini_{\text{padre}} = 1 - (0,5^2 + 0,5^2) = 1 - 0,5 = 0,5$$

$$Gini_{\text{izq}} = 1 - (0,6^2 + 0,4^2) = 1 - 0,52 = 0,48$$

$$Gini_{\text{der}} = 1 - (0,4^2 + 0,6^2) = 1 - 0,52 = 0,48$$

$$Gini_{\text{hijos}} = \frac{100}{200} \times 0,48 + \frac{100}{200} \times 0,48 = 0,48$$

Gini detecta una mejora (de 0,5 a 0,48), lo que permite al árbol seguir creciendo hacia nodos más puros. Esto se debe a que Gini es sensible a los cambios en las proporciones de las clases, mientras que *Misclassification Error* solo se fija en cuál es la clase mayoritaria, lo que lo hace inadecuado como métrica de entrenamiento.

Ejercicio 11

Si se agregan o se elimina un pequeño porcentaje de las muestras, la estructura del árbol puede cambiar completamente, porque el mismo se construye de forma *greedy*: cada nodo elige el mejor *split* posible en ese momento. Por eso se dice que el modelo tiene una alta variabilidad y sensibilidad ante un cambio sutil en los datos.

Ejercicio 12

Si llega $X = 75$, esta observación caerá lo más a la derecha posible del árbol. Incluso, tendrá el mismo tratamiento que aquellas observaciones muy cercanas a $X = 50$. Esto pasa porque el árbol fue entrenado con valores de X entre 0 y 50, por lo que todas las reglas de partición van a tener *thresholds* dentro de ese rango. Es una limitación importante de los árboles de decisión.

Ejercicio 13

Explicarle un modelo de regresión logística puede parecer abstracto para alguien que no esté metido en el tema. Por otro lado, un árbol de decisión de baja profundidad modela perfectamente la toma de decisiones que tenemos los humanos, por lo que es más transparente y más fácil de explicar.

Ensamblados

Ejercicio 14

Puede haber dos casos en el cual Majority Voting arroje un resultado: dos modelos aciertan o tres modelos aciertan. Esto se puede representar como:

$$P(\text{dos aciertan}) = 0,8 \cdot 0,8 \cdot (1 - 0,8) = 0,512$$

$$P(\text{tres aciertan}) = 0,8 \cdot 0,8 \cdot 0,8 = 0,512$$

Son dos casos disjuntos en los que el ensamble puede acertar, por lo que:

$$P(\text{ensamble acierta}) = 0,512 + 0,512 = 0,896$$

O sea, *Accuracy* subió de 0,8 a 0,896.

Ejercicio 15

Rank Correlation se usa en lugar de Pearson porque los árboles de decisión trabajan con *rankings* y *thresholds*, no con relaciones lineales.

Pearson mide la relación lineal entre dos variables. Si dos variables tienen una relación monótona pero no lineal, Pearson puede dar un valor bajo aunque en la práctica contengan información muy similar para un árbol.

En cambio, **Rank Correlation** mide la relación monótona: solo le importa si cuando una variable sube, la otra también sube (o baja), sin importar si la relación es lineal o no. Y esto es exactamente lo que le importa a un árbol de decisión, porque al hacer *splits* del tipo $X \leq \text{threshold}$, lo único que importa es el orden de los valores, no su magnitud. Si dos variables tienen el mismo *ranking* (el mismo orden de las observaciones), van a producir exactamente los mismos *splits* y, por lo tanto, son redundantes para el modelo.

Ejercicio 16

- **Paralelizables.** Son aquellos en los cuales cada árbol se entrena de forma independiente sobre su propio subconjunto de datos. Ejemplos: **Bagging** y **Random Forest**.
- **No paralelizables.** Son aquellos en los cuales cada árbol nuevo se entrena para corregir los errores del anterior. Se necesita el resultado del árbol i para poder entrenar el árbol $i + 1$. Ejemplos: **AdaBoost**, **XGBoost**, y cualquier otro método de *boosting*.

Ejercicio 17

Hay dos métodos de interpretabilidad para medir la importancia de los atributos:

1. **Feature Importance.** Se calcula promediando las reducciones de *Gini* que produce cada atributo en todos los *splits* de todos los árboles del bosque. Los atributos que generan mayores reducciones son las más importantes.
2. **Permutation Importance.** En lugar de mirar las reducciones de *Gini*, mide cuánto empeora la *performance* del modelo al desordenar aleatoriamente los valores de cada atributo en el conjunto de test.